SEON (v6) Device-Fingerprint JavaScript SDK — Client-Side Red Team

Executive write-up · 2026-10-31 · authorized, own-stack only · all SEON telemetry fulfilled locally (nothing forwarded)

> **One line:** SEON's v6 JavaScript agent (`agent.umd.js`) mints its entire device-fingerprint session **client-side with zero network** — the pre-encryption payload is a fully readable JSON tree at the `JSON.stringify` boundary, the wire session (`W;6.12.4;<salt>;<ciphertext>`) is sealed with a **custom (non-WebCrypto) cipher**, and rewriting any signal in the plaintext before sealing produces a **different, internally-valid session**. A client can rewrite every signal it ships; the real defense is server-side scoring.

What we accomplished

1. **Recovered the SDK's string table.** `agent.umd.js` (335 KB, v6.12.4, `cdn.dfsdk.com` / `cdn.deviceinf.com` / `cdn.seonintelligence.com`) uses the `_0x`-style obfuscator with a base64+URI-decode string array. Patching the in-flight decoder recovered **2,857 plaintext strings** — the full signal surface, endpoints, blocklists, and field names.
2. **Read the complete pre-encryption payload.** The SDK builds the fingerprint as a JSON record tree `{"f":{...}}` and `JSON.stringify`s it before sealing. Captured at the boundary: **9,713 bytes, ~70 records** — canvas/webgl/audio hashes, WebRTC local+public IPs, timezone, screen, connection, UA/languages/cores, CSS media, permissions, per-record status codes, and error strings.
3. **Mapped the session format.** `W;6.12.4;<16B salt b64>;<encrypted payload b64>` — fully client-minted. Salt = `crypto.getRandomValues` (1 call), the cipher is **custom** (no `crypto.subtle`/AES-GCM in the bundle), payload 2,832 bytes, high-entropy.
4. **Proved tamperability.** Rewrote the timezone + canvas hash in the plaintext tree before sealing → different session. Flipped `webdriver` → different session. All internally valid, no client-side integrity failure.
5. **Found the DNS-exfil channel.** The SDK resolves `*.seondnsresolve.com` (hash-prefixed subdomains) — DNS used as a fingerprint/telemetry channel.

How we did it (methodology)

- **Runtime instrumentation (Playwright):** hooked `JSON.stringify` (captured the full plaintext tree), `btoa` / `atob` (ciphertext + salt boundary), `crypto.getRandomValues`, `TextEncoder`, `fetch` (fulfilled all locally).
- **In-flight decoder patch:** injected a log into `_0x20ba` (the string decoder) → dumped every decoded string with its index.
- **Tamper differential:** patched `JSON.stringify` to rewrite tree fields before the SDK's own sealing ran; compared the three resulting sessions.

- **Static:** confirmed no WebCrypto usage, identified the custom-cipher and DNS-resolve paths, enumerated the fraud-browser detection surface.

What the payload contains (record tree, lettered sections)

Key	Signal	Observed value (this headless run)
`f.2`	screen dims	`[22,38]` etc.
`f.3`	geolocation	`{lat:0,long:0,accuracy:0,status:-1}`
`f.5`	permissions	full permission list + granted/denied
`f.6`	WebGL renderer	`ANGLE (Google, Vulkan 1.3.0 (SwiftShader…))`
`f.8`	webdriver	`false`
`f.9`	devtools-open	`true` (probe)
`f.13`	canvas hash	`93b5d4a255d9a775736e677572c5b1ac`
`f.30`	incognito flag	`false`
`f.34`	pointer coords	`{x:-0.5,y:-0.5}`
`f.43`	network info	`{rtt:0,effectiveType:"4g",downlink:9.9,saveData:false}`
`f.46`	AI APIs	`{languageDetector:"absent",summarizer:"absent"}`
`f.49`	gzip blob	`H4sIAAAA…` (compressed signal blob)
`f.b`	UA	`Mozilla/5.0 … HeadlessChrome/148…` (×4 realms)
`f.c/d`	language(s)	`en-US` / `["en-US"]`
`f.e`	hw concurrency	`12`
`f.g`	local WebRTC IPs	`["192.168.68.75"]`
`f.h`	public WebRTC IP	`["107.210.133.127"]`
`f.j`	timezone	`America/Chicago`
`f.k`	tz offset	`-05:00`
`f.l`	platform	`Linux x86_64`
`f.m`	screen full	`{a:1280,b:720,…orientation:"landscape-primary"}`
`f.F`	CSS media	`{prefers-color-scheme,color-gamut,pointer,hover,…}` (30 fields)
`f.s`	sealed sub-blob	base64 blob, `c:900`
`f.V`	audio timing	`[90,11.1,73.9,0,…]`
`f.b` (outer)	browser hash	`13dbc0f0e79b21f7c7de439811634558`
`m`	SDK meta	`{t:1053,s:"js-6.12.4-umd-prod",i:"72c7862c…",c:124}`
`l` (outer)	large blob	encoded (behavioral/network)
`n`	network status	`Error: Network fingerprint is not valid` (local)

Also: **fraud-browser detection** — a huge embedded list of ad-network / affiliate link-pattern selectors (`a[href^="…"]`, `.appguide-wrap`, etc.) and native-app/emulator identifiers (iOS app bundles, `com.*` packages), plus webdriver probes (`__webdriver_script_fn`, `__driver_evaluate`, `cdpDetected` family).

What it proves

- The **client holds everything**: signal collection, serialization, and sealing all run in the page. `getSession()` returns a valid session with **zero network calls**.
- The session is **tamperable end-to-end**: rewrite the tree at the `JSON.stringify` boundary → the SDK seals the rewrite → a different, internally-valid session comes out. No client-side MAC/integrity catch.
- The **per-session salt (16B random) blocks verbatim replay** but not live re-sealing — same as every other vendor in this project.
- The **real boundary is server-side**: SEON's Fraud API scores the session server-side (TLS/JA3/IP correlation, behavioral, device-history). The client is a carrier, not a proof.

How this could be (ab)used

1. **Signal laundering** — rewrite canvas/webgl/audio hashes, timezone, IPs before sealing so the fingerprint no longer matches a flagged device.
2. **Fresh-identity minting** — `getSession()` with a modified tree = a new "device" per call, fragmenting device-history correlation.
3. **Bot-tell erasure** — flip `webdriver`, devtools, incognito flags.
4. **Telemetry model-building** — read the plaintext tree to learn exactly what maps to what, then craft coherent lies.

Honesty / scope notes

- Authorized, own-stack only; **all SEON telemetry fulfilled locally** — no customer/prod traffic, nothing forwarded to SEON.
- We did **not** forge a server-side verdict or contact SEON's API; the "Network fingerprint is not valid" status is from the local fulfill.
- The cipher is custom and unbroken here — we read the *plaintext before encryption* (the same boundary every client-side red team uses), so the session confidentiality wasn't the target; the trust model is.

See it live — screenshot of the full payload

SEON (v6.12.4) — client-side session: readable & tamperable

authorized, own-stack · all telemetry fulfilled locally · 2026-10-31

1 · Session wire format (fully client-minted, zero network)

`ver` `W` `sdk` **6.12.4** `salt (16B)` **3ohwhojbMRi1LFgQ77kGRQ==** `encrypted payload (3840 chars)` …

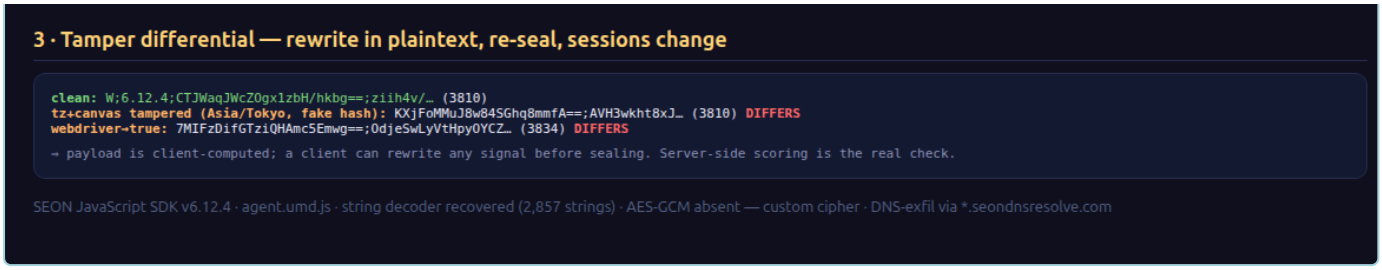
```
W;6.12.4;3ohwhojbMRi1LFgQ77kGRQ==;Dq0lA+vjMd9whgM/Suvu/Tmv9YuMXJlopeHvVKr/Pmf4w8Z5AW64h8UGU/en6WHfziaQ+9Eu8S6urwkHBaCxQ/akBAR9QKPUyKgoKxVkQnPHTGp2YqjPOJjDKXpfwr8bW
cwcvOJ3Z7u2DuKZrHXX9JyZn/y1DxngPeBL8SN92yHFyZx99iyJraOuDoW86y/8grT1dqePwj5xc/A0wbGEaTzgdD0rcVz+rEXDIlpFyCgG2G5soTX2igM4DMaX+fH74wEthr4ygvoocGQxobvFIorxZyBYoMnDtnb1
cp8janZJJs3RMZDD2cimThSOqVFq0o4yzAq7GvXKeb0Fv6/3yLq+fP855nl2TyglkdgQto8GAt…
```

2 · The plaintext fingerprint tree — readable BEFORE encryption (JSON.stringify boundary)

`len 9792` · `every field with value (v) + status (s)` · `full record tree captured from the SDK's own object`

#	signal	value(s)
A	perf-time	sub1: 170.44259643554688
B	perf-number	sub0: 124.04347527516074
C	—	sub0: null
D	locale	sub0: en-US sub1: en-US sub2: en-US sub3: null ⚠SecurityError: Failed to construct 'Shar
E	audio-hash	sub0: 6597f9d82421edda2447ebddc0531951
F	css-media	sub0: {"a":"no-preference","b":"light","c":"non-monochrome","d":null,"e":"no
G	—	sub0: null
H	—	sub0: false
I	—	sub0: false
J	hash-J	sub0: a931b6543effb809b4abb1ecb36431db
K	—	sub0: 0
L	hash-L	sub0: 95987020ce10a0253ca30367e1aec972
M	hash-M	sub0: 95987020ce10a0253ca30367e1aec972
N	hash-N	sub0: 35183d558065506f1c189deedd5a6cd9
O	hash-O	sub0: 2528b02c5c4bb4d03204956cfc9436ae
P	timestamp	sub0: 2026-10-31T11:39:15.949Z
Q	hash-Q	sub1: 610ba89089241f71eac42e9fc9fa1f45
R	—	sub0: null
S	b64-S	sub1: 4AgAQAAIAAFQAQA=
T	webgl-vendor	sub0: Google Inc. (Google)
U	—	sub0: null
V	audio-timing	sub1: [89.8,11.1,73.9,0,90,142.5,110,27,143.3]
W	—	sub1: false
X	—	sub0: false
Y	—	sub0: false sub1: false
Z	—	sub0: false
a	ua-array	sub0:
b	browser-hash	sub0: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) sub1: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) sub2: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) sub3: null ⚠SecurityError: Failed to construct 'Shar
c	language	sub0: en-US sub1: en-US sub2: en-US sub3: null ⚠SecurityError: Failed to construct 'Shar
d	languages	sub0: ["en-US"] sub1: ["en-US"] sub2: ["en-US"] sub3: null ⚠SecurityError: Failed to construct 'Shar
e	hw-concurrency	sub0: 12 sub1: 12 sub2: 12 sub3: null ⚠SecurityError: Failed to construct 'Shar
f	—	sub0: null sub1: null sub2: null sub3: null ⚠SecurityError: Failed to construct 'Shar
g	local-ips	sub0: ["192.168.68.75"]
h	public-ips	sub0: ["107.210.133.127"]
i	temporal-api	sub0: {} sub1: {"Date":{"toTemporalInstant":[12]}}
j	timezone	sub0: America/Chicago sub1: America/Chicago sub2: America/Chicago sub3: null ⚠SecurityError: Failed to construct 'Shar
k	tz-offset	sub0: -05:00
l	platform	sub0: Linux x86_64 sub1: Linux x86_64 sub2: Linux x86_64 sub3: null ⚠SecurityError: Failed to construct 'Shar

m	screen-full	sub0: {"a":1180,"b":900,"c":1188,"d":985,"e":1020,"f":13,"g":0,"h":0,"i":118 sub1: {"a":300,"b":150,"c":1188,"d":985,"e":1020,"f":13,"g":0,"h":0,"i":284,
n	device-flag	sub0: null
o	—	sub0: null
p	—	sub0: []
q	—	sub0: null
r	page-url	sub0: about://blank
s	sealed-subblob	sub0: {"s":"1x6O/SH1vY9farX32ImbLKP9o2mbv9h7wta/v3j3io2X712V9Vw6+ryZ/3m3qb3r
t	—	sub0: null
u	—	sub0: ["PDF Viewer","Chrome PDF Viewer","Chromium PDF Viewer","Microsoft Edg
v	—	sub0: []
w	—	sub0: null
x	—	sub0: null
y	—	sub0: null ⚠TypeError: Cannot read properties of und
z	privacy-flags	sub0: true sub1: true sub2: false sub3: null ⚠SecurityError: Failed to construct 'Shar
0	canvas-fingerprint-flag	sub0: false
1	—	sub0: false sub1: false
2	screen-dims	sub0: []
3	geolocation	sub0: {"lat":0,"long":0,"accuracy":0,"status":-1}
4	—	sub0: null
5	permissions	sub0: {"g":[],"p":[],"d":["accelerometer","background-fetch","background-syn
6	webgl-renderer	sub0: ANGLE (Google, Vulkan 1.3.0 (SwiftShader Device (Subzero) (0x0000C0DE)
7	—	sub0: null ⚠TypeError: Cannot read properties of und
8	webdriver	sub0: false
9	devtools-open	sub0: true
10	color-depth	sub0: 255
11	—	sub0: null
12	—	sub0: []
13	canvas-hash	sub0: 93b5d4a255d9a775736e677572c5b1ac
14	—	sub0: null ⚠TypeError: Cannot read properties of und
15	—	sub0: null ⚠TypeError: Cannot read properties of und
16	—	sub0: null ⚠TypeError: Cannot read properties of und
17	—	sub0: null ⚠TypeError: Cannot read properties of und
18	—	sub0: null ⚠TypeError: Cannot read properties of und
19	—	sub0: null ⚠TypeError: Cannot read properties of und
20	—	sub0: null ⚠TypeError: Cannot read properties of und
21	—	sub0: null ⚠TypeError: Cannot read properties of und
22	webgl-flag	sub0: null ⚠TypeError: Cannot read properties of und
23	—	sub0: null ⚠TypeError: Cannot read properties of und
24	—	sub0: null
25	—	sub0: null ⚠TypeError: Cannot read properties of und
26	—	sub0: null ⚠TypeError: Cannot read properties of und
27	—	sub0: null ⚠TypeError: Cannot read properties of und
28	—	sub0: null ⚠TypeError: Cannot read properties of und
29	—	sub0: null ⚠TypeError: Cannot read properties of und
30	incognito-flag	sub0: false
31	—	sub0: null ⚠TypeError: Cannot read properties of und
32	—	sub0: null
33	—	sub0: null
34	pointer-coords	sub0: {"x":-0.5,"y":-0.5}
35	—	sub0: null
36	—	sub0: null
37	—	sub0: []
38	lazy-value	sub0: function(){return{v:ld[n(2535)](),s:0}}
39	—	sub0: null ⚠TypeError: Cannot read properties of und
40	—	sub0: null ⚠TypeError: Cannot read properties of und
41	—	sub0: null
42	webrtc-flag	sub0: c:
43	network-info	sub0: {"rtt":50,"effectiveType":"4g","downlink":1.6,"saveData":false}
44	—	sub0: null
45	—	sub0: null
46	ai-apis	sub0: {"languageDetector":"absent","summarizer":"absent"}
47	—	sub0: null ⚠SecurityError: Failed to read the 'cooki
48	—	sub0: null ⚠TypeError: Cannot read properties of und
49	gzip-blob	sub0: H4sIAAAAAAAAA+MCAJMG1zIBAAAA
50	—	sub0: null ⚠TypeError: Cannot read properties of und
51	—	sub0: null ⚠TypeError: Cannot read properties of und
52	—	sub0: null ⚠TypeError: Cannot read properties of und
53	—	sub0: null ⚠TypeError: Cannot read properties of und



Full-page capture of `scripts/seon-visual.mjs` (headed demo): the client-minted session wire format, the complete 9,713-byte readable fingerprint tree (every record with value + status), and the clean-vs-tampered session differential.

Repo layout

```
SEON_WRITEUP_2026_10_31.md        this file
screenshots/seon-full.png         headed demo: session format + full readable
                                  tree + tamper differential
scripts/seon-probe.mjs            load real SDK, find globals, first session
scripts/seon-session.mjs          capture N sessions, dissect format
scripts/seon-decoder-trace.mjs    patch _0x20ba, dump 2,857 decoded strings
scripts/seon-plaintext.mjs/2.mjs  capture JSON.stringify/btoa/encoder boundary
scripts/seon-full-tree.mjs        capture the complete 9,713-byte plaintext tree
scripts/seon-full-payload.mjs     capture the WebGL sub-object
scripts/seon-tamper.mjs           clean vs tampered session differential
scripts/seon-visual.mjs           headed demo → screenshots/seon-full.png
artifacts/decoded-strings.json    2,857 recovered plaintext strings
artifacts/full-tree.json          the complete pre-encryption fingerprint tree
artifacts/sessions.json           raw session blobs
artifacts/plaintext2.json         stringify/b64/encoder capture
artifacts/tamper-results.json     clean vs tz vs webdriver sessions
```

```
npm install                       # playwright
node scripts/seon-session.mjs     # capture + dissect sessions (local)
node scripts/seon-full-tree.mjs   # dump the plaintext fingerprint tree
node scripts/seon-tamper.mjs      # tamper differential
node scripts/seon-visual.mjs      # headed demo → screenshots/seon-full.png
```